

江西七校联盟入门赛第三场题解

A - 斐波那契平方数

求出范围内的所有斐波那契数，再判断是否为完全平方数即可。或者使用结论：斐波那契数列仅有 1, 144 是完全平方数。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

signed main() {
    int x;
    cin >> x;
    int a = 1, b = 1;
    int ans = 2;
    while (a + b <= x) {
        int t = b;
        b = a + b;
        a = t;
        int f = (int)(sqrtl(b) + 0.5);
        if (f * f == b) ans++;
    }
    cout << ans << endl;
    return 0;
}
```

B - 魏征进谏

按题目要求排序即可。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

struct node {
    string a, b;
};

signed main() {
    int n;
    cin >> n;
    vector<node> a(n);
    unordered_map<string, int> mpa = {"High", 2}, {"Medium", 1}, {"Low", 0};
    unordered_map<string, int> mpb = {"Military", 3}, {"Livelihood", 2},
    {"Governance", 1}, {"PersonalCultivation", 0};
    for (int i = 0; i < n; i++) {
        cin >> a[i].a >> a[i].b;
    }
}
```

```

sort(a.begin(), a.end(), [&](node& x, node& y) {
    if (mpa[x.a] != mpa[y.a]) return mpa[x.a] > mpa[y.a];
    return mpb[x.b] > mpb[y.b];
});
for (int i = 0; i < n; i++) {
    cout << a[i].a << ' ' << a[i].b << endl;
}
return 0;
}

```

C - 哈基值

可以从后往前遍历数组，维护后缀最大值，从而 $O(1)$ 判断是否需要乘以 -1 。

参考代码

```

#include <bits/stdc++.h>
using namespace std;

signed main() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    int mx = a[n];
    long long ans = a[n];
    for (int i = n - 1; i >= 1; i--) {
        ans += a[i] * (mx > a[i] ? -1 : 1);
        mx = max(mx, a[i]);
    }
    cout << ans << endl;
    return 0;
}

```

D - 全排列

将填数顺序变成从大到小即可。

参考代码

```

#include <iostream>
using namespace std;
const int N = 10;

int n, a[N], vis[N];

void dfs(int deep) {
    if (deep == n) {
        for (int i = 0; i < n; i++) cout << a[i] << ' ';
        cout << endl;
        return;
    }
    for (int i = n; i >= 1; i--) {
        if (!vis[i]) {

```

```

        vis[i] = 1;
        a[deep] = i;
        dfs(deep + 1);
        vis[i] = 0;
    }
}

int main() {
    cin >> n;
    dfs(0);
    return 0;
}

```

E - Monsters

[Problem - 1849B - Codeforces](#)

[CF1849B Monsters - 洛谷](#)

F - 加一或乘二

设 $F(x)$ 为数组中前 x 个元素含有的递增子序列的个数，那么易知 $F(1) = 2$ 。下面我们探讨 $F(x)$ 和 $F(x - 1)$ 之间的关系。

1. 当 $a[x] = 0$ 时， $a[x]$ 不能和前面的任何子序列连接形成递增子序列，除了空序列。所以此时 $F(x) = F(x - 1) + 1$ 。
2. 当 $a[x] = 1 + \max_{1 \leq j \leq x-1} a_j$ 时，对于前面所有的递增子序列，加上 $a[x]$ 后依然合法，故有 $F(x) = 2 \times F(x - 1)$ 。

事实上题目已经给出了答案。

参考代码

```

#include <bits/stdc++.h>
using namespace std;
const int mod = 1e9 + 7;

signed main() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    int ans = 2;
    for (int i = 2; i <= n; i++) {
        if (a[i] > a[i - 1]) ans = 2 * ans % mod;
        else ans = (ans + 1) % mod;
    }
    cout << ans << endl;
    return 0;
}

```

G - 求和

题目灵感来源: [P2388 阶乘之乘 - 洛谷](#)

推导过程, 其中 $f(x) = \frac{x(x+1)}{2}$:

$$\begin{aligned}\sum_{i=1}^n \sum_{j=1}^{+\infty} \lfloor \frac{i}{m^j} \rfloor &= \sum_{j=1}^{+\infty} \sum_{i=1}^n \lfloor \frac{i}{m^j} \rfloor \\&= \sum_{z=m^j, j \in N_+}^{z \leq n} \sum_{i=1}^n \lfloor \frac{i}{z} \rfloor \\&= \sum_{z=m^j, j \in N_+}^{z \leq n} \sum_{i=1}^n \frac{i - i \% z}{z} \\&= \sum_{z=m^j, j \in N_+}^{z \leq n} \frac{1}{z} \left(f(n) - \sum_{i=1}^n i \% z \right) \\&= \sum_{z=m^j, j \in N_+}^{z \leq n} \frac{1}{z} \left(f(n) - \lfloor \frac{n}{z} \rfloor \times f(z-1) - f(n \% z) \right)\end{aligned}$$

暴力解决时间复杂度为 $O(n \log n)$, 其中外层求和 $O(n)$, 内层求和 $O(\log n)$ 。通过交换求和顺序, 然后将内层求和的 $O(n)$ 优化为 $O(1)$, 从而使问题在 $O(\log n)$ 的复杂度解决。

参考代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int f(int x) {
    return x * (x + 1) / 2;
}

signed main() {
    int n, m;
    cin >> n >> m;
    int sum = f(n);
    int ans = 0;
    for (int j = m; j <= n; j *= m) {
        int he = sum - n / j * f(j - 1) - f(n % j);
        ans += he / j;
    }
    cout << ans << endl;
    return 0;
}
```

H - 两极反转

首先, 为了减少我们的分析难度, 我们可以先考虑在只有操作 2 的情况下是否能使两个数 x, y 相同, 同时也是为了方便, 将 $32bit$ 降低到 $8bit$ 考虑。

需要发现的是, 将连续的 1 视作一体后, 操作 2 不可能将这两个连续的 1 变成 1 个连续的 1, 也不可能变成 3 个, 这是因为操作 2 的条件要求修改的那一位前后不同, 但是我们可以任意延长或缩短这连续的 1 形成的区域块。

观察到这一点之后，答案就比较明显了。 x 能够变成 y 需要有完全相同的 0, 1 块，例如 0011 0011 和 0101 111，将连续的 0 和连续 1 视为一体后，是同样的 0101 的格式。

然后我们再来处理可以取反所有位的操作，明显的是，块 0101 是可以变成 1010 的，意味着只要这样分块看后，块的个数相同，那么就一定可以变成相同的数。

再来处理区间询问的问题，我这里直接使用 $x = y$ 来表示 x 可以变成 y 。可以发现，成为好数组的条件是 $x_1 = x_2 = x_3 = \dots$ 只要区间内出现一个 $\neq$ 那么就不可能成为好的数组。所以可以利用等式的传递性，用 e_i 来表示 arr_{i-1} 是否 $= arr_i$ ，我用 $e_i = 0$ 视为 $=$ ，否则视为 $\neq$ 。只需要检验 e 数组区间内是否存在 1 即可，可以利用前缀和检验。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
const int N = 2e5+5;
#define int long long

int ca[N];
int cnt[N];
int ans[N];

int cal(int x) {
    int res = 0;
    for (int i = 0; i < 32; i++) {
        int j = i;
        while (j < 32 && (x >> i & 1) == (x >> j & 1)) j++;
        res++;
        i = j - 1;
    }
    return res;
}

void solve() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; i++) cin >> ca[i];
    for (int i = 1; i <= n; i++) cnt[i] = cal(ca[i]);

    for (int i = 2; i <= n; i++) {
        ans[i] = ans[i - 1];
        ans[i] += cnt[i] != cnt[i - 1];
    }

    while (q--) {
        int l, r;
        cin >> l >> r;
        int res = ans[r] - ans[l];
        if (res) cout << "No" << endl;
        else cout << "Yes" << endl;
    }
}

signed main() {
```

```
solve();  
return 0;  
}
```